

WorldContact: A Contact-Centric World Model for Scalable Robot Learning

Caoliwen Wang¹, Mengdi Wang^{†,1}, Heng Zhang¹,
Shixun Huang¹, Siyuan Chen¹, Chao Liu¹, Anpei Chen²,
Zhendong Wang³, Peter Yichen Chen^{†,1}, Huamin Wang³

Abstract: Adapting robots to new objects and tasks requires interaction experience that can be costly to obtain. We present WorldContact, a contact-centric world model for deformable-object manipulation, constructed from a limited set of high-quality trajectories to generate additional training data efficiently. It predicts object dynamics using larger time steps than the source numerical simulator, which requires small integration steps to resolve rapid motion and prevent interpenetration. We evaluate WorldContact across 16 shopping-bag manipulation tasks. State-rollout measurements on a single H100 GPU show a $10\times$ speedup over the source simulator, excluding rendering and disk I/O. We use the generated data to fine-tune an existing vision-language-action policy and deploy it directly on a real robot. In bag lifting, the same policy achieves 65% single-attempt success when fine-tuned on source simulation data alone, compared with 95% when fine-tuned on the dataset expanded with WorldContact. These results support efficient data generation with WorldContact for robot policy adaptation.

I. INTRODUCTION

A central goal of embodied AI is to develop robots that can learn and act across diverse tasks and environments. Household services, industrial production, and logistics all require adaptation to changing objects, materials, task requirements, and physical interactions. Learning reusable manipulation skills from diverse experience can help robots acquire these capabilities [1], [2]. To act reliably in these settings, robots must anticipate how their actions will affect the world. A *world model* describes this relationship by predicting the outcomes of actions given the current world state. Such predictive models provide a basis for reasoning about physical interactions [3], [4].

Adapting robot policies to unfamiliar scenes requires relevant interaction experience. World models can help supply this experience by predicting the outcomes of robot-object interactions. Constructing suitable world models is challenging because objects differ in geometry, material properties, and physical behavior. For deformable-object manipulation, a world model must predict how objects deform and move in response to robot actions. Yet changing shapes continually alter physical interactions, from friction between fabric folds to sliding on support surfaces and robot grasping. Modeling

these interactions across changing configurations makes reliable prediction difficult [5]–[7].

To address this modeling challenge, we introduce *WorldContact*, a contact-centric neural simulator that serves as a world model for deformable-object manipulation. It builds on the Lagrangian approach of WorldParticle [8] to learn object dynamics from a limited set of high-quality interaction trajectories and rapidly generate additional interaction trajectories for robot policy learning. The high-quality source trajectories may come from physics-based numerical simulation or real-world interaction. It encodes local geometry and motion around object vertices to represent interactions within and between objects and with the robot and environment, enabling prediction of object dynamics under prescribed robot actions.

To support efficient data generation, WorldContact uses a time step that is $20\times$ the source simulator’s integration step. Numerical simulation often requires small steps to resolve rapid motion and prevent interpenetration; WorldContact predicts the resulting state change over the larger interval directly. Combined with GPU-efficient inference, this larger time step reduces the cost of generating policy-training trajectories, with some approximation of high-accuracy dynamics. In our measurements, WorldContact generates state rollouts $10\times$ faster than the source simulator, excluding rendering and disk I/O. We demonstrate this approach on 16 robotic shopping-bag manipulation tasks, including lifting and supporting a bag. We fine-tune an existing pretrained vision-language-action (VLA) policy on the expanded simulation dataset and deploy it directly on a real robot. In the bag-lifting evaluation, the same VLA succeeds in 13/20 single-attempt trials (65%) when fine-tuned only on the 25 source simulation trajectories, compared with 19/20 trials (95%) when fine-tuned on the expanded dataset. These results demonstrate the practical value of our data-scaling approach: efficiently generating additional training experience from limited high-quality source data to adapt robot control policies to new manipulation settings.

Our contributions are:

- WorldContact, a contact-centric world model constructed from limited high-quality data that encodes local geometry and motion to predict object dynamics under robot actions.
- A data-generation pipeline that renders predicted trajectories and pairs the observations with their conditioning

¹ University of British Columbia.

² Westlake University.

³ Style3D Research.

[†] Corresponding author.

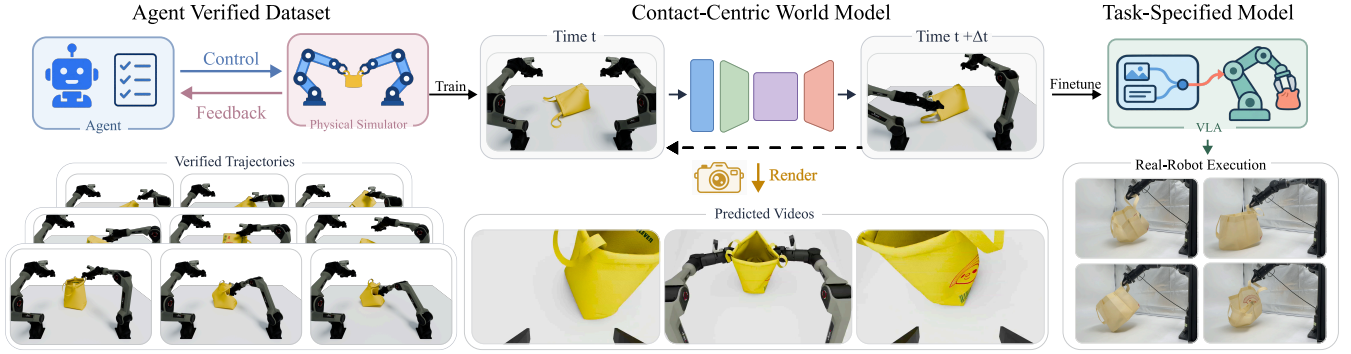


Fig. 1. **WorldContact overview.** (Left): an agent-verified physical-simulation dataset, where an automated simulation agent samples the object and prescribed state and executes the resulting controls in the physical simulator. (Middle): contact-centric world model prediction and rendering of object-state trajectories into the observation modality consumed by the VLA. (Right): VLA fine-tuning for task-specific policy adaptation and subsequent real-robot deployment.

robot actions to scale demonstrations for policy fine-tuning.

- A shopping-bag study across 16 tasks, demonstrating $10\times$ faster state-rollout generation and better real-robot bag-lifting performance with the expanded dataset than with source data alone.

II. RELATED WORK

a) World Models for Robot Learning: Ha and Schmidhuber [3] explored policy learning using latent world models. Dreamer [4] learns control policies through imagined trajectories in a learned latent space. For cloth, VisuoSpatial Foresight learns RGB-D dynamics in simulation and uses predicted observations to plan multi-step smoothing and folding [9]. For robotic manipulation, IRASim generates action-conditioned videos [10], while Interactive World Simulator uses learned visual rollouts for policy training and evaluation, including tasks involving deformable objects [11]. Together, these works use learned rollouts for prediction, planning, policy optimization, and demonstration collection. Their image or latent predictions do not directly provide the material-point correspondences and boundary-relative geometry needed for our vertex-error and penetration metrics. Our method maintains explicit vertex states for these checks and renders observations after state prediction, separating dynamics validation from observation generation.

b) Neural Dynamics for Deformable Manipulation: DPI-Net and GNS represent physical systems as interacting particles and learn their dynamics through graph message passing [12], [13]. MeshGraphNets extends learned simulation to mesh representations, including cloth with adaptive resolution [14]. For cloth smoothing, Visible Connectivity Dynamics infers connectivity between observed points and learns dynamics on this graph to plan under partial observability [15]. GraphGarment learns action-conditioned garment dynamics for bimanual manipulation and uses a residual model for sim-to-real correction [16]. Diffusion Dynamics Models jointly address full cloth-state reconstruction from partial observations and action-conditioned prediction for model-predictive folding [17]. RoboCraft learns parti-

cle dynamics from RGB-D observations for elastoplastic shaping [18], while RoboCook uses learned particle dynamics for long-horizon manipulation of elastoplastic objects with tools [19]. Subsequent work addresses material adaptation with AdaptiGraph [20], learning from RGB-D interaction videos with Particle-Grid Neural Dynamics [21], and multiple objects and materials with ParticleFormer [22]. PointWorld represents scene motion and robot actions as 3D point flows to learn manipulation dynamics across embodiments [23]. SoMA learns robot-conditioned soft-body dynamics over reconstructed 3D Gaussian splats from real observations [24]. Contact-Aware Neural Dynamics uses tactile contact information to improve forward prediction and sim-to-real policy alignment in rigid-object grasping [25]. These approaches motivate explicit interaction modeling, while differing in supervision, state representation, and access to contact measurements. Our method uses full vertex-state supervision from a calibrated teacher and explicitly encodes spatial, topological, controllable, and environmental contact.

WorldParticle models Lagrangian particle dynamics with a transformer that combines local interaction encoding and global communication [8]. WorldContact adopts this architectural basis to condition object dynamics on prescribed robot motion and environmental geometry. Our contact-centric formulation models interactions among objects, the robot, and the environment, including contact through rigidly held objects. We evaluate the resulting predictions through dynamics fidelity and their use in training a real-robot policy.

c) Simulation and Sim-to-Real for Deformable Manipulation: Visual domain randomization addresses observation differences [26], while simulation adaptation uses real rollouts to adjust the distribution of simulated dynamics [27]. SoftGym provides deformable manipulation tasks for robot learning [5]; GarmentLab and DexGarmentLab expand garment assets, interactions, and manipulation benchmarks [28], [29]. DexGarmentLab also uses garment correspondences to generate trajectories from a single expert demonstration. FlingBot learns dynamic cloth unfolding [30], while Cloth Funnels combines dynamic and quasi-static actions to

canonicalize and align garments for downstream tasks [31]. Alongside these environments and policies, MimicGen expands demonstrations by transforming and executing source trajectories [32], and SoftMimicGen extends demonstration generation to deformable objects [6]. SimWeaver combines measurement-backed deformable simulation, trajectory synthesis, and sensor-aware variation in rendered observations for RGB sim-to-real transfer without per-task calibration [7]. These systems address the availability and diversity of simulated experience. Our work uses a learned world model to reuse existing state trajectories for further interaction-data generation. In the simulation-seeded implementation evaluated here, calibrated physical rollouts supply supervision, and model rollouts generate additional trajectories. Teacher calibration and coverage therefore remain relevant to this setting.

At the solver and asset level, DiffCloth differentiates through dry frictional contact for cloth parameter estimation and control [33]. DiffCP identifies cloth material parameters from RGB-D observations and uses the calibrated model for state estimation [34]. PhysTwin reconstructs deformable digital twins from sparse interaction videos, combining spring-mass dynamics with Gaussian-splat rendering [35]. Our evaluated implementation uses calibrated assets and studies the learned transition used for subsequent rollout generation.

III. METHOD

We represent the scene using two complementary components: the object state, which jointly describes all objects whose dynamics are to be predicted, and the prescribed state, which specifies robot motion and external geometry supplied as inputs. At step t , the object state is represented as

$$\mathcal{S}_t = [\mathbf{X}_t, \mathbf{V}_t, \mathcal{E}, \mathbf{C}], \quad (1)$$

where $\mathbf{X}_t, \mathbf{V}_t \in \mathbb{R}^{N \times 3}$ denote the vertex positions and velocities, respectively, and $\mathcal{E}$ represents the rest topology of the target objects. For vertex i , c_i concatenates an object-identity tag with its rest-state (u,v) coordinates, which specify its location in the object’s topological space. The attributes of all predicted vertices are stacked to form $\mathbf{C} \in \mathbb{R}^{N \times 3}$.

The prescribed state is represented as

$$\mathcal{B}_t = [\mathbf{a}_t, \mathbf{G}, \mathbf{X}_t^E, \mathbf{V}_t^E, \mathbf{C}_t^E], \quad (2)$$

where $\mathbf{a}_t = [\mathbf{q}_t^L, d_t^L, \mathbf{q}_t^R, d_t^R] \in \mathbb{R}^{14}$, $\mathbf{q}_t^L, \mathbf{q}_t^R \in \mathbb{R}^6$ are commanded joint rotations for the two arms, and d_t^L, d_t^R are their gripper openings. $\mathbf{G}$ specifies the fixed kinematic structure and the geometries of its links, gripper, and objects rigidly held by the robot. $\mathbf{X}_t^E \in \mathbb{R}^{N_t^E \times 3}$ and velocities $\mathbf{V}_t^E \in \mathbb{R}^{N_t^E \times 3}$ represent positions and velocities of sampled environmental geometry, such as the table. The environmental attributes $\mathbf{C}_t^E \in \mathbb{R}^{N_t^E \times 4}$ concatenate environment tags and surface normals. Each environment tag identifies the geometry to which the sample point belongs. An object is included in $\mathcal{S}_t$ if its state is predicted by the world model, and in $\mathcal{B}_t$ if its state is prescribed externally. For example, a ball released by the robot is included in $\mathcal{S}_t$ because its subsequent motion must be predicted, whereas a ball rigidly

held by the robot is included in $\mathcal{B}_t$ because its motion is prescribed by the robot.

Our model learns how the object state evolves under contact with the prescribed state:

$$(\Delta \mathbf{X}_t, \Delta \mathbf{V}_t) = \mathcal{F}_\theta(\mathcal{S}_t, \mathcal{B}_{t+\Delta t}). \quad (3)$$

Then we update the target next state as $\widehat{\mathbf{X}}_{t+\Delta t} = \mathbf{X}_t + \Delta \mathbf{X}_t$ and $\widehat{\mathbf{V}}_{t+\Delta t} = \mathbf{V}_t + \Delta \mathbf{V}_t$. Figure 2 shows the model architecture.

A. Contact-Centric Vertex Tokenizer

For each target vertex i , we encode the local geometry and motion associated with the following contacts:

- *Spatial contact.* $\mathcal{N}_i^S$ contains vertices that are close to i in Euclidean space across all target objects. It captures contacts between different parts or objects, such as between a bag’s strap and body or between two predicted objects.
- *Topological contact.* $\mathcal{N}_i^T$ contains vertices that are nearby in the rest-state topology $\mathcal{E}$ of the same object, represented in UV space. It captures local interactions between vertices connected through the object’s intrinsic topology.
- *Controllable contact.* $\mathcal{N}_i^C$ contains nearby samples from robot links, grippers, and objects rigidly held by the robot. It captures direct robot–object contact as well as contact mediated by held objects, such as a rigid tool pushing the bag.
- *Environmental contact.* $\mathcal{N}_i^E$ contains nearby samples from environmental geometry, such as a table supporting the bag.

We denote the positions, velocities, and attributes of controllable samples at time t by $\mathbf{X}_t^C, \mathbf{V}_t^C, \mathbf{C}_t^C$. Controllable samples follow the commanded configuration $\mathbf{a}_t$ and kinematic geometry $\mathbf{G}$. Their attributes $\mathbf{C}_t^C$ use the tags and surface normals as $\mathbf{C}_t^E$. For $k \in \{S, T, C, E\}$, a compact-support learned kernel aggregates neighbor features:

$$\mathbf{g}_i^k = \sum_{j \in \mathcal{N}_i^k} \mathbf{W}_k(\mathbf{r}_{ij}^k) \mathbf{u}_{ij}^k. \quad (4)$$

The corresponding displacements and features are

$$\begin{aligned} \text{Spatial Contact: } \mathbf{r}_{ij}^S &= \mathbf{x}_{t,j} - \mathbf{x}_{t,i}, & \mathbf{u}_{ij}^S &= [\mathbf{v}_{t,j}, c_j], \\ \text{Topology Contact: } \mathbf{r}_{ij}^T &= \mathbf{x}_{t,j} - \mathbf{x}_{t,i}, & \mathbf{u}_{ij}^T &= [\mathbf{v}_{t,j}, c_j], \\ \text{Controllable Contact: } \mathbf{r}_{ij}^C &= \mathbf{x}_{t+\Delta t,j}^C - \mathbf{x}_{t,i}, & \mathbf{u}_{ij}^C &= [\mathbf{v}_{t+\Delta t,j}^C, c_{t+\Delta t,j}^C], \\ \text{Environmental Contact: } \mathbf{r}_{ij}^E &= \mathbf{x}_{t+\Delta t,j}^E - \mathbf{x}_{t,i}, & \mathbf{u}_{ij}^E &= [\mathbf{v}_{t+\Delta t,j}^E, c_{t+\Delta t,j}^E]. \end{aligned} \quad (5)$$

The resulting vertex token is

$$\mathbf{h}_i = \text{MLP}([\mathbf{x}_{t,i}, \mathbf{v}_{t,i}, c_i, \mathbf{g}_i^S, \mathbf{g}_i^T, \mathbf{g}_i^E, \mathbf{g}_i^C]). \quad (6)$$

Each token thus combines the target vertex’s state and attributes with local geometric features and contact information.

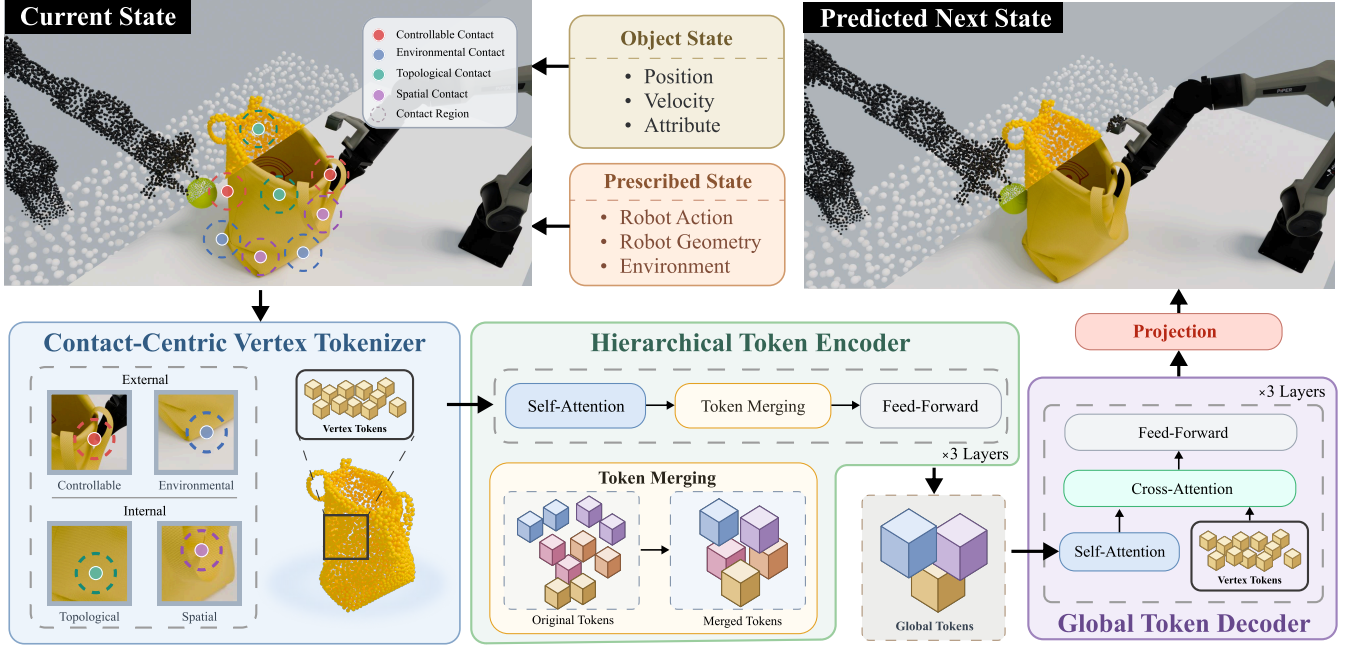


Fig. 2. **WorldContact pipeline.** Given the current object state and prescribed state, WorldContact proceeds through four stages: the contact-centric vertex tokenizer encodes spatial, topological, controllable, and environmental contact information into vertex tokens; the hierarchical encoder merges vertex tokens into global tokens for long-range communication; and the global-token decoder refines vertex tokens to produce vertex-level state corrections. A differentiable projection is used to improve long-horizon rollout stability during fine-tuning.

B. Hierarchical Token Encoder

The vertex tokens $\mathbf{h}_i$ effectively encode local geometric information and external contact interactions. However, many physical phenomena in deformable objects exhibit long-range coupling. For example, when a cloth is stretched by pulling one region, vertices that are spatially distant from each other may undergo similar motions because they experience similar deformation and force patterns. To model such long-range coupling, self-attention provides a natural mechanism for propagating information across distant vertices [36]. Applying self-attention repeatedly over all N vertices, however, is computationally expensive. We adopt WorldParticle’s hierarchical token-merging strategy [8] to reduce the number of tokens involved in global attention.

Let $\mathbf{H}_e^{(0)} = [\mathbf{h}_i]_{i=1}^N$, $\mathbf{X}^{(0)} = \mathbf{X}_t$, and $m_i^{(0)} = 1$. Each encoder level first exchanges global context at its current resolution using multi-head self-attention [36] with 3D rotary positional encoding (RoPE) [8]:

$$\widehat{\mathbf{H}}_e^{(\ell)} = \text{SelfAttn}(\mathbf{H}_e^{(\ell-1)}, \mathbf{X}^{(\ell-1)}). \quad (7)$$

Tokens are then divided into two alternating partitions. Each token in the first is matched to the token in the second with the highest cosine similarity, reducing the count to $N^{(\ell)} = \lceil N^{(\ell-1)}/2 \rceil$. If $\mathcal{M}_i^{(\ell)}$ is the set assigned to output token i ,

the merged feature, anchor, and multiplicity are

$$m_i^{(\ell)} = \sum_{j \in \mathcal{M}_i^{(\ell)}} m_j^{(\ell-1)},$$

$$(\mathbf{h}_i^{(\ell)}, \mathbf{x}_i^{(\ell)}) = \sum_{j \in \mathcal{M}_i^{(\ell)}} \frac{m_j^{(\ell-1)}}{m_i^{(\ell)}} (\widehat{\mathbf{h}}_j^{(\ell)}, \mathbf{x}_j^{(\ell-1)}). \quad (8)$$

Repeated merging yields $(\overline{\mathbf{H}}, \overline{\mathbf{X}}) = (\mathbf{H}_e^{(L_e)}, \mathbf{X}^{(L_e)})$ with $N_g \ll N$ global tokens. Each global token summarizes the local geometric, contact, velocity, and attribute information of the vertex group it represents. Together, these tokens form a compact representation of the current object state that supports efficient information sharing across spatially distant regions.

C. Global Token Decoder

The original vertex tokens bypass compression and initialize the queries $\mathbf{H}_d^{(0)} = \mathbf{H}_e^{(0)}$; the global tokens start from $\mathbf{H}_g^{(0)} = \overline{\mathbf{H}}$. Subsequent self-attention operates on these global tokens to capture long-range dependencies:

$$\mathbf{H}_g^{(\ell)} = \text{SelfAttn}(\mathbf{H}_g^{(\ell-1)}, \overline{\mathbf{X}}). \quad (9)$$

The updated global representation is then queried by the vertex-resolution tokens through cross-attention:

$$\mathbf{H}_d^{(\ell)} = \text{CrossAttn}(\mathbf{H}_d^{(\ell-1)}, \mathbf{X}_t; \mathbf{H}_g^{(\ell)}, \overline{\mathbf{X}}), \quad (10)$$

which allows each vertex query to retrieve global context relevant to its local state while retaining the local geometric

and contact information encoded in the vertex tokens. A pointwise output head predicts the corrections:

$$(\Delta \mathbf{x}_{t,i}, \Delta \mathbf{v}_{t,i}) = \text{MLP}_{\text{out}}(\mathbf{h}_{d,i}^{(L_d)}). \quad (11)$$

D. Training

We learn WorldContact from high-quality interaction trajectories, using their recorded states as supervision. During training, we sample trajectory segments of length W and autoregressively roll out the model over the segment. The training objective is computed over the predicted states at all rollout steps, averaging the position and velocity errors across all predicted vertices:

$$\mathcal{L} = \frac{1}{N(W-1)} \sum_{n=1}^{W-1} \sum_{i=1}^N (\lambda_x \|\hat{\mathbf{x}}_{t+n\Delta t,i} - \mathbf{x}_{t+n\Delta t,i}\|_2^2 + \lambda_v \|\hat{\mathbf{v}}_{t+n\Delta t,i} - \mathbf{v}_{t+n\Delta t,i}\|_2^2). \quad (12)$$

Gradients propagate through all rollout steps to reduce error accumulation during autoregressive prediction.

Starting from the base model, we continue training with differentiable topology projection $\mathcal{P}$ inside the autoregressive prediction loop to improve long-horizon stability. Let $\tilde{\mathbf{X}}_{t+\Delta t}$ denote the unconstrained position predicted by the network. The state used by the rollout is

$$\hat{\mathbf{X}}_{t+\Delta t} = \mathcal{P}(\tilde{\mathbf{X}}_{t+\Delta t}, \mathcal{E}), \quad \hat{\mathbf{V}}_{t+\Delta t} = \frac{\hat{\mathbf{X}}_{t+\Delta t} - \hat{\mathbf{X}}_t}{\Delta t}. \quad (13)$$

The projection smooths the predicted displacement field and limits topology-edge distortion. Its output is used in Eq. (12) and fed into the next autoregressive step, allowing gradients to propagate through both the projection iterations and the full rollout.

IV. EXPERIMENTS

A. Data Collection

We develop an automated simulation agent powered by GPT-5.6 Sol with xhigh reasoning effort to configure and run tasks in the physical simulator used by SimWeaver [7]. The agent monitors the resulting trajectories and discards executions that fail the checks described below. To align simulated dynamics with real-world behavior, we first reconstructed each target object as a digital asset and calibrated its simulation parameters using real-world observations. We discretize the target bag into 20,274 vertices, while rigid spheres and cubes use 951 and 465 vertices, respectively. The robot gripper and table are represented by 1,114 and 16,900 sampled points, respectively. Simulator parameters and solver settings are fixed across the dataset, with a 0.002s physical time step chosen to ensure numerical stability; object-specific parameters, including friction, damping, and Young’s modulus, are fixed for each object to be predicted. For each execution, the agent first samples the initial object state. It then partitions feasible object configurations and interaction locations into coverage cells, visits these cells while varying the interaction speed and motion amplitude,

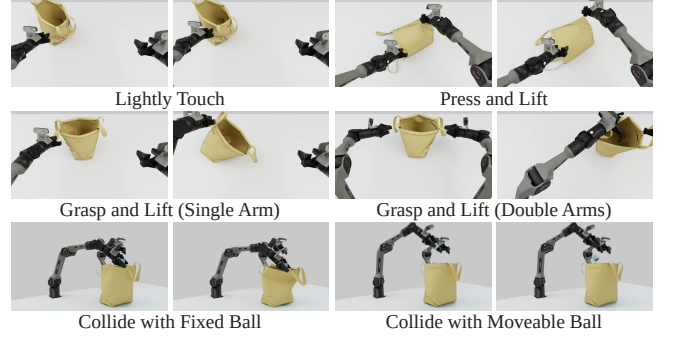


Fig. 3. **Action and contact diversity in the collected corpus.** Each panel shows a representative shopping-bag task, including object collision, grasping, lifting, placing, and releasing, covering complex contacts.

and records the resulting trajectory together with its conditioning action. The dataset covers 16 shopping-bag manipulation and collision tasks, including object collision, grasping, lifting, placing, and releasing. Together, these tasks involve spatial, topological, controllable, and environmental contacts, including simultaneous interactions, with a subset shown in Fig. 3.

Each trajectory passes online physical and visual checks. Physical checks reject states containing undefined or infinite values (NaN or Inf), solver failures, interpenetration, self-intersection, and spurious gripper–bag adhesion. Visual checks reject executions that move the bag outside the workspace or camera view. The state machine is RESET → EXECUTE → VALIDATE → ACCEPT; a failed check terminates the rollout and logs its first failing predicate. Only trajectories that pass every check are used for training.

The agent maintains a memory of past execution outcomes and failure cases, allowing it to learn from previous failures and avoid repeating similar failures in future executions.

B. Training Details

We use the same dataset split for initial WorldContact training and subsequent training with differentiable topology projection inside the prediction loop (Sec. III-D). The dataset covers 16 shopping-bag manipulation tasks, with 25 training trajectories per task (400 in total), including two validation and two test trajectories per task. Both training stages use 8 NVIDIA H100 GPUs, five-step rollout segments with a time step of 0.04 s, and equal position and velocity loss weights of 10^4 . Further training details and model specifications are provided in the Appendix.

C. Neural Dynamics Comparison

We compare WorldContact with three neural dynamics baselines: RoboCraft [18], RoboCook [19], and Adapti-Graph [20], shown in Fig. 4. All methods are trained on the same dataset with identical dataset splits. For autoregressive test rollouts, we report cloth vertex position and velocity mean squared error (MSE) against ground truth, averaged over all vertices and frames. The penetration rate (%) measures the percentage of cloth contact area penetrating the

TABLE I
NEURAL DYNAMICS COMPARISON.

Method	Position MSE ↓	Velocity MSE ↓	Penetration Rate (%) ↓
RoboCraft [18]	0.0598	0.0106	13.88%
RoboCook [19]	0.0575	0.0105	10.28%
AdaptiGraph [20]	0.0323	0.0098	24.84%
WorldContact	0.0002	0.0002	0.35%

TABLE II
TRAINING TIME PER OPTIMIZATION STEP (S).

Attention Type	$W = 5 \downarrow$	$W = 10 \downarrow$	$W = 15 \downarrow$
w/o hierarchical token compression	4.367	11.236	15.873
w/ hierarchical token compression	4.184	10.526	14.493

collision geometry by more than 1 cm, weighted by vertex-associated mesh areas and aggregated across all frames. Table I reports the aggregate test results; lower values are better for all metrics.

D. Computational Efficiency

To evaluate computational efficiency, we compare complete state rollouts generated by WorldContact and the physical simulator used for data generation, using the same initial object state and prescribed-state inputs. All measurements are performed on a single NVIDIA H100 GPU, excluding rendering and disk I/O. WorldContact simulates 0.4 s of dynamics per second of wall-clock time, compared with 0.04 s for the physical simulator, yielding a $10\times$ speedup.

E. Ablation Study

a) *Contact-centric vertex tokenizer*: We ablate the contact-centric vertex tokenizer. For each vertex i , we remove the spatial, topological, controllable, and environmental contact features $\mathbf{g}_i^S$, $\mathbf{g}_i^T$, $\mathbf{g}_i^C$, and $\mathbf{g}_i^E$, and form its token by linearly projecting the concatenation of $[\mathbf{x}_{t,i}, \mathbf{v}_{t,i}, \mathbf{c}_i]$ with the features of its nearest environmental and controllable sample into the same dimension as $\mathbf{h}_i$. The downstream architecture remains unchanged. We train the new architecture from scratch using exactly the same training strategy throughout. As shown in Fig. 5, removing these contact features leads to vertex penetration into the table and gripper, resulting in unstable rollouts and contact failures, while using our tokenizer maintains stable contacts.

b) *Hierarchical token compression*: We ablate the hierarchical token compression paradigm of our method. Specifically, we replace the hierarchical token encoder and global token decoder with full-resolution self-attention blocks with the same number of attention layers. We report the estimated training time per optimization step for training segment lengths $W = 5, 10, 15$ on a single H100 GPU. Table II shows that global-token communication reduces the time per optimization step while preserving the vertex-level output; lower values are better.

V. APPLICATION

We fine-tune an existing VLA policy for single-arm bag lifting using a larger dataset produced with WorldContact

from the initial high-quality simulation data. Given an initial object state and a sequence of prescribed-state inputs, the world model predicts an object-state trajectory, which is then rendered into the observation modality consumed by the VLA [37]. Each rendered observation is paired with the corresponding control command used to condition the rollout, ensuring consistent observation-action alignment. For this evaluation, we use 25 high-quality simulated bag-lifting trajectories as source data and set the expanded policy-training dataset size to 150 trajectories. We adopt a right-arm-only control interface and provide the policy with two RGB views captured by a top-mounted camera and a right-wrist-mounted camera. The language instruction is “right grasp and lift.” Starting from the pretrained Pi05 checkpoint, we fine-tune the policy for the target task. Details are provided in Appendix.

After fine-tuning, we freeze the policy and deploy it directly on the real robot. The task requires a single attempt to grasp the bag, lift it above the table, and hold it for 2 seconds without human intervention. The same VLA fine-tuned only on the 25 source trajectories succeeds in 13 of 20 trials (65%). Fine-tuning with the expanded dataset increases success to 19 of 20 trials (95%). This task-level comparison supports the usefulness of the generated data for policy adaptation.

Figure 6 illustrates our real-world experimental setup, including the robot platform, the bag used as the target object, and the camera configuration. It also shows a representative successful execution sequence, demonstrating the complete process from approaching and grasping the bag to lifting it above the table and maintaining the lifted state.

VI. CONCLUSION

We presented WorldContact, a contact-centric world model constructed from limited high-quality interaction data to generate training experience efficiently for robot learning. Its larger time step and GPU-efficient inference enable state-rollout generation at $10\times$ the speed of the source numerical simulator. We evaluated WorldContact across 16 shopping-bag manipulation tasks and used its generated data for VLA fine-tuning. For real-robot bag lifting, the same VLA achieved 65% single-attempt success when fine-tuned on source simulation data alone, compared with 95% when fine-tuned on the dataset expanded with WorldContact. These results support efficient data generation for robot policy adaptation.

Prediction errors under distribution shift and differences between rendered and real observations remain challenges. Future work will examine how source-data coverage, prediction fidelity, and generated-data budgets affect downstream policy performance. Policy failures could guide the collection of additional real or simulated interactions and subsequent world-model updates. Building WorldContact from real-world interaction data and evaluating transfer to unseen objects, materials, and tasks will further test its potential for scalable robot learning.

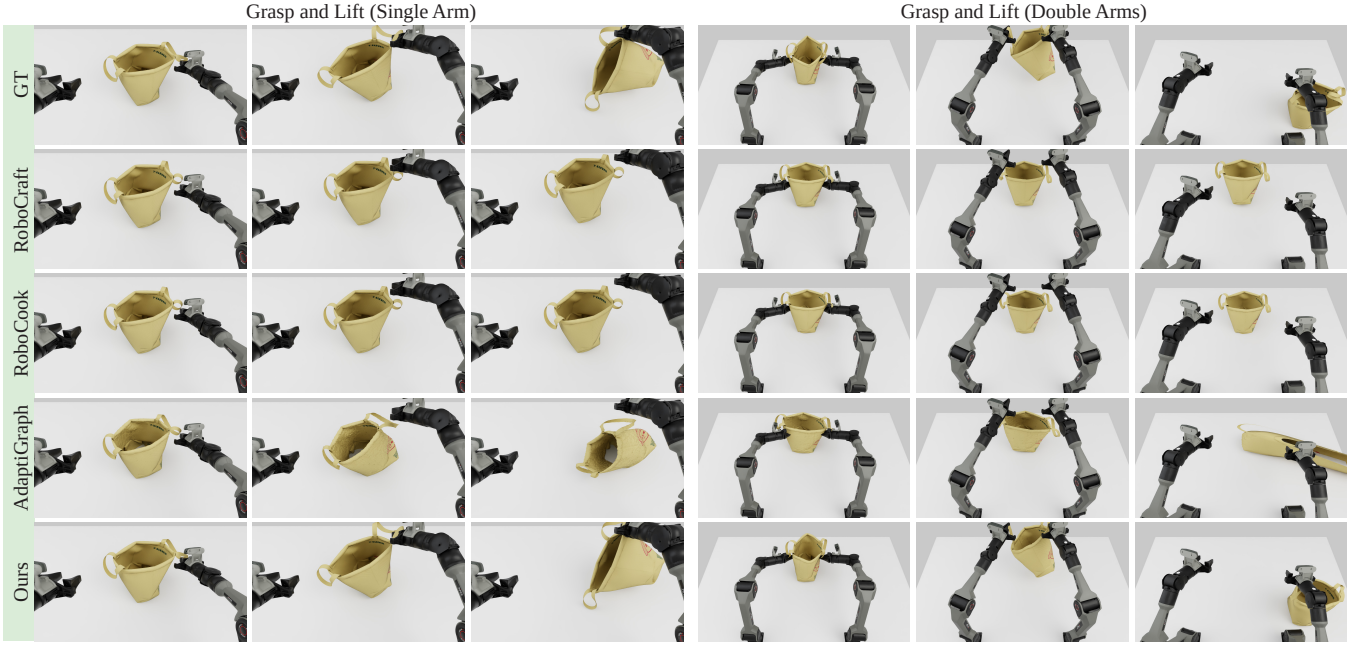


Fig. 4. **Qualitative rollout comparison.** The top row shows the ground-truth states, while the subsequent rows show rollouts from RoboCraft [18], RoboCook [19], AdaptiGraph [20], and Ours. Columns correspond to successive time steps. RoboCraft [18] and RoboCook [19] fail to maintain the bag–gripper contact during the lifting process, causing the bag to remain stationary. AdaptiGraph suffers from severe physically implausible deformation. In contrast, our method successfully simulates the lifting process while maintaining stable rollouts. The rendered views on the left are consistent with those used for VLA fine-tuning.

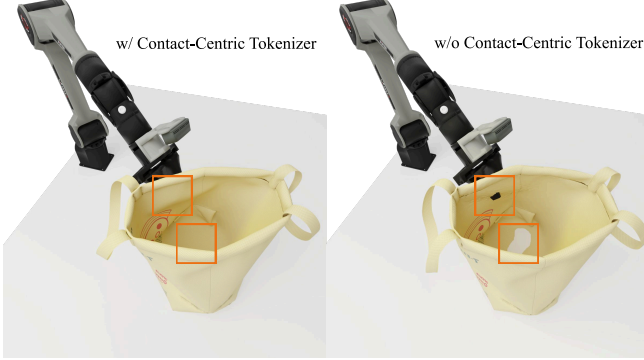


Fig. 5. **Ablation of the contact-centric vertex tokenizer.** Same state with (left) and without (right) the contact-centric vertex tokenizer. Orange boxes highlight the corresponding contact regions, where removing our tokenizer leads to penetration.

APPENDIX

a) WorldContact Specification: The particle tokenizer uses $4 \times 4 \times 4$ continuous-convolution kernels with width 64. Fluid and topology features each have 32 channels, while boundary and direct particle features each have 64 channels, yielding $\mathbf{h}_i \in \mathbb{R}^{192}$. The super-token encoder performs $L_e = 3$ self-attention and bipartite token-merging iterations with 8 heads (head dimension 24, $k = 2$) and 3D RoPE of dimension 24. The decoder contains $L_d = 3$ cross/self-attention layers at width 192, with 8 heads, FFN width 128, and dropout 0.1. The prediction head is a $192 \rightarrow 128 \rightarrow 128 \rightarrow 6$ MLP. The tokenizer, super-token encoder, decoder,

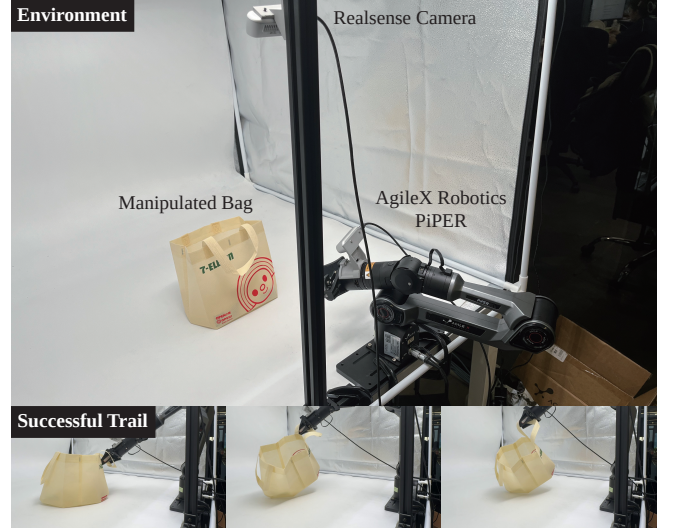


Fig. 6. **Real world experiment.** The top row shows our experimental setup, including the AgileX PiPER 6-DoF robotic arm, the manipulated 7-Eleven bag, and two RealSense cameras: a D435i top-mounted camera and a D435 right-wrist-mounted camera. The bottom row shows a successful trial.

and prediction head contain 57,993, 782,784, 1,116,480, and 42,502 trainable parameters, respectively, for a total of 1,999,759 parameters (approximately 2.00M).

b) WorldContact Training Details: We use AdamW with $\beta = (0.9, 0.999)$ and a weight decay of 5×10^{-4} . Base training uses a peak learning rate of 1×10^{-4} , with an 8,000-step linear warmup from 1×10^{-6} followed by cosine

decay toward 5×10^{-6} , and runs for 93,240 optimization steps. Starting from the pretrained model, continued training with differentiable topology projection uses a peak learning rate of 2×10^{-5} , with an 8,000-step linear warmup from 2×10^{-7} followed by cosine decay toward 5×10^{-6} , and runs for 31,080 optimization steps. After each predicted step, we apply a differentiable projection with four Laplacian-smoothing iterations (weight 0.35), followed by 32 edge-constraint iterations that restrict each edge length to 0.9–1.1 times its rest-topology length.

c) Policy Fine-Tuning Details: We use AdamW with $\beta = (0.9, 0.95)$, $\epsilon = 1 \times 10^{-8}$, and weight decay 1×10^{-2} . Training uses an initial learning rate of 2.5×10^{-5} , with a 1,000-step linear warmup followed by cosine decay to 2.5×10^{-6} over a total of 30,000 optimization steps. State and action features are normalized using quantile normalization, while visual features use identity normalization. The policy predicts 50 future action steps per chunk, and we perform full-model fine-tuning starting from the pretrained checkpoint.

REFERENCES

- [1] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Chormanski, T. Ding, D. Driess, A. Dubey, C. Finn, P. Florence, C. Fu, M. G. Arenas, K. Gopalakrishnan, K. Han, K. Hausman, A. Herzog, J. Hsu, B. Ichter, A. Irpan, N. Joshi, R. Julian, D. Kalashnikov, Y. Kuang, I. Leal, L. Lee, T.-W. E. Lee, S. Levine, Y. Lu, H. Michalewski, I. Mordatch, K. Pertsch, K. Rao, K. Reymann, M. Ryoo, G. Salazar, P. Sanketi, P. Sermanet, J. Singh, A. Singh, R. Soricut, H. Tran, V. Vanhoucke, Q. Vuong, A. Wahid, S. Welker, P. Wohlhart, J. Wu, F. Xia, T. Xiao, P. Xu, S. Xu, T. Yu, and B. Zitkovich, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *CoRL*, 2023.
- [2] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, Q. Vuong, T. Kollar, B. Burchfiel, R. Tedrake, D. Sadigh, S. Levine, P. Liang, and C. Finn, “Openvla: An open-source vision-language-action model,” in *CoRL*, 2024.
- [3] D. Ha and J. Schmidhuber, “Recurrent world models facilitate policy evolution,” in *NeurIPS*, 2018.
- [4] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” in *ICLR*, 2020.
- [5] X. Lin, Y. Wang, J. Olkin, and D. Held, “SoftGym: Benchmarking deep reinforcement learning for deformable object manipulation,” in *CoRL*, 2021.
- [6] M. Moghani, M. Azizian, A. Garg, Y. Zhu, S. Huver, and A. Mandlekar, “SoftMimicGen: A data generation system for scalable robot learning in deformable object manipulation,” in *ICRA*, 2026.
- [7] W. Hu, H. Wang, Y. Li, L. Liu, M. Zhao, L. Jiang, X. Tang, J. Wei, Z. Shu, Z. Wang, Z. Su, H. Wang, and R. Yang, “SimWeaver: Zero-shot RGB sim-to-real for deformable manipulation,” *arXiv preprint arXiv:2606.15338*, 2026.
- [8] C. Wang, M. Guo, S. Chen, H. Zhang, M. Wang, X. Ni, H. Sun, K. Wang, Z. Pan, K. Wu, L. Liu, Y. Yang, C. Jiang, T. Komura, W. Matusik, and P. Y. Chen, “Worldparticle: Unified world simulation of lagrangian particle dynamics via transformer,” *arXiv preprint arXiv:2605.15305*, 2026.
- [9] R. Hoque, D. Seita, A. Balakrishna, A. Ganapathi, A. K. Tanwani, N. Jamali, K. Yamane, S. Iba, and K. Goldberg, “VisuoSpatial Foresight for multi-step, multi-task fabric manipulation,” in *RSS*, 2020.
- [10] F. Zhu, H. Wu, S. Guo, Y. Liu, C. Cheang, and T. Kong, “IRASim: A fine-grained world model for robot manipulation,” in *ICCV*, 2025.
- [11] Y. Wang, R. Syed, F. Wu, M. Zhang, A. Onol, J. Barreiros, H. Nayyeri, T. Dear, H. Zhang, and Y. Li, “Interactive world simulator for robot policy training and evaluation,” in *RSS*, 2026.
- [12] Y. Li, J. Wu, R. Tedrake, J. B. Tenenbaum, and A. Torralba, “Learning particle dynamics for manipulating rigid bodies, deformable objects, and fluids,” in *ICLR*, 2019.
- [13] A. Sanchez-Gonzalez, J. Godwin, T. Pfaff, R. Ying, J. Leskovec, and P. Battaglia, “Learning to simulate complex physics with graph networks,” in *ICML*, 2020.
- [14] T. Pfaff, M. Fortunato, A. Sanchez-Gonzalez, and P. W. Battaglia, “Learning mesh-based simulation with graph networks,” in *ICLR*, 2021.
- [15] X. Lin, Y. Wang, Z. Huang, and D. Held, “Learning visible connectivity dynamics for cloth smoothing,” in *CoRL*, 2022.
- [16] W. Chen, K. Li, D. Lee, X. Chen, R. Zong, and P. Kormushev, “Graph-Garment: Learning garment dynamics for bimanual cloth manipulation tasks,” in *IROS*, 2025.
- [17] T. Tian, H. Li, B. Ai, X. Yuan, Z. Huang, and H. Su, “Diffusion dynamics models with generative state estimation for cloth manipulation,” in *CoRL*, 2025.
- [18] H. Shi, H. Xu, Z. Huang, Y. Li, and J. Wu, “RoboCraft: Learning to see, simulate, and shape elasto-plastic objects with graph networks,” in *RSS*, 2022.
- [19] H. Shi, H. Xu, S. Clarke, Y. Li, and J. Wu, “RoboCook: Long-horizon elasto-plastic object manipulation with diverse tools,” in *CoRL*, 2023.
- [20] K. Zhang, B. Li, K. Hauser, and Y. Li, “AdaptiGraph: Material-adaptive graph-based neural dynamics for robotic manipulation,” in *RSS*, 2024.
- [21] K. Zhang, B. Li, K. Hauser, and Y. Li, “Particle-grid neural dynamics for learning deformable object models from RGB-D videos,” in *RSS*, 2025.
- [22] S. Huang, Q. Chen, X. Zhang, J. Sun, and M. Schwager, “ParticleFormer: A 3D point cloud world model for multi-object, multi-material robotic manipulation,” in *CoRL*, 2025.
- [23] W. Huang, Y.-W. Chao, A. Mousavian, M.-Y. Liu, D. Fox, K. Mo, and L. Fei-Fei, “PointWorld: Scaling 3D world models for in-the-wild robotic manipulation,” in *CVPR*, 2026.
- [24] M. Huang, H. Wang, K. Ren, L. Xu, Y. Zhou, M. Yu, B. Dai, and J. Pang, “SoMA: A real-to-sim neural simulator for robotic soft-body manipulation,” in *ICML*, 2026.
- [25] C. Jing, J. K. Bandi, J. Ye, Y. Duan, P. Abbeel, X. Wang, and S. Yi, “Contact-aware neural dynamics,” in *CVPR*, 2026.
- [26] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *IROS*, 2017.
- [27] Y. Chebotar, A. Handa, V. Makoviychuk, M. Macklin, J. Issac, N. Ratliff, and D. Fox, “Closing the sim-to-real loop: Adapting simulation randomization with real world experience,” in *ICRA*, 2019.
- [28] H. Lu, R. Wu, Y. Li, S. Li, Z. Zhu, C. Ning, Y. Shen, L. Luo, Y. Chen, and H. Dong, “GarmentLab: A unified simulation and benchmark for garment manipulation,” in *NeurIPS*, 2024.
- [29] Y. Wang, R. Wu, Y. Chen, J. Wang, J. Liang, Z. Zhu, H. Geng, J. Malik, P. Abbeel, and H. Dong, “DexGarmentLab: Dexterous garment manipulation environment with generalizable policy,” in *NeurIPS*, 2025.
- [30] H. Ha and S. Song, “FlingBot: The unreasonable effectiveness of dynamic manipulation for cloth unfolding,” in *CoRL*, 2022.
- [31] A. Canberk, C. Chi, H. Ha, B. Burchfiel, E. Cousineau, S. Feng, and S. Song, “Cloth Funnels: Canonicalized-alignment for multi-purpose garment manipulation,” in *ICRA*, 2023.
- [32] A. Mandlekar, S. Nasiriany, B. Wen, I. Akinola, Y. Narang, L. Fan, Y. Zhu, and D. Fox, “MimicGen: A data generation system for scalable robot learning using human demonstrations,” in *CoRL*, 2023.
- [33] Y. Li, T. Du, K. Wu, J. Xu, and W. Matusik, “DiffCloth: Differentiable cloth simulation with dry frictional contact,” *ACM Trans. Graph.*, vol. 42, no. 1, pp. 2:1–2:20, 2023.
- [34] D. Zheng, S. Yao, W. Xu, and C. Lu, “Differentiable cloth parameter identification and state estimation in manipulation,” *IEEE Robot. Autom. Lett.*, vol. 9, no. 3, pp. 2519–2526, 2024.
- [35] H. Jiang, H.-Y. Hsu, K. Zhang, H.-N. Yu, S. Wang, and Y. Li, “Phys-Twin: Physics-informed reconstruction and simulation of deformable objects from videos,” in *ICCV*, 2025.
- [36] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in *NeurIPS*, 2017.
- [37] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, S. Jakubczak, T. Jones, L. Ke, S. Levine, A. Li-Bell, M. Mothukuri, S. Nair, K. Pertsch, L. X. Shi, J. Tanner, Q. Vuong, A. Walling, H. Wang, and U. Zhilinsky, “ π_0 : A vision-language-action flow model for general robot control,” in *RSS*, 2025.